Design Document

Vibe Code Security Scanner

Version: 1.0
Author: Jude Perera
Project: Vibe Code Security Scanner
Date: August 2026

1. Application Architecture

1.1 System Overview

The application follows a three-tier architecture: a static HTML frontend, a Python FastAPI backend, and a SQLite database. The frontend communicates with the backend via REST API calls. The backend coordinates scanning and persists results.

Layer	Technology	Responsibility
Frontend	HTML, CSS, JavaScript	User interface, form submission, results display
Backend	Python, FastAPI	API routes, repo cloning, scan coordination
Database	SQLite	Scan history and findings persistence
Scanner	Python AST + Regex	Security check execution

1.2 Data Flow

- User enters GitHub URL in the web form and clicks Run Scan
- Frontend sends POST /scan with the repo_url in the request body
- Backend validates the URL and clones the repo to a temp directory
- Analyser walks all files and runs applicable checks per file type
- Findings collected and saved to the database via save_scan() and save_finding()
- Backend returns scan_id, total_findings, vulnerability_rate, and findings list
- Frontend renders findings in severity-ranked table
- Temp directory deleted — repo clone no longer exists on disk

---

2. Screen Design

2.1 Dashboard — Main Screen

The main screen is a single page divided into three sections arranged vertically.

Section 1 — Header

- Application name: Vibe Code Security Scanner
- Subtitle: Automated security analysis for AI-generated codebases
- Dark background (#0f172a), white text
- Full width, fixed at top of page

Section 2 — Scan Input

- Large text input field: placeholder 'https://github.com/username/repo'
- Run Scan button — blue (#3b82f6), full width on mobile
- Loading indicator shown while scan runs: 'Scanning repository...'
- Error message shown in red if scan fails
- Status message shown on completion: 'Scan complete for [repo URL]'

Section 3 — Results

- Four summary cards: Total Findings, High Severity, Medium Severity, Low Severity
- Findings table with columns: Severity badge, Check Name, File Path, Line Number, Detail
- Severity badges: red pill for HIGH, amber for MEDIUM, green for LOW

- Table rows alternate light grey and white for readability
- Empty state: 'No findings to display' when scan returns clean

2.2 Scan History Panel

- Located below the results section
- Heading: Scan History
- Each history entry shows: repo URL (truncated), scan date, total findings count
- Clicking a history entry loads that scan's findings into the results table
- Most recent scan shown first
- Empty state: 'No previous scans' on first load

2.3 Navigation and Connections

User Action	Result
Enter URL + click Run Scan	POST /scan → results appear in findings table
Click history entry	GET /scans/{id} → findings load into results table
Page load	GET /scans → history panel populated automatically
Scan error	Error message shown below input, results table cleared

2.4 Colour Palette

Element	Colour	Hex
Background	Dark navy	#0f172a
Card background	Dark slate	#1e293b
HIGH severity	Red	#ef4444
MEDIUM severity	Amber	#f59e0b
LOW severity	Green	#22c55e
Primary button	Blue	#3b82f6
Body text	White	#ffffff
Secondary text	Grey	#94a3b8

---

3. API Design

3.1 Endpoints

Method	Path	Description	Response
GET	/	Serves the HTML dashboard	HTML page
POST	/scan	Clones repo, runs scan, saves results	JSON: scan_id, findings, total, rate
GET	/scans	Returns all scan history	JSON array of scans
GET	/scans/{scan_id}	Returns findings for a specific scan	JSON array of findings

3.2 POST /scan Request Body

{ "repo_url": "https://github.com/username/repo" }

3.3 POST /scan Response

- scan_id: integer — database ID of this scan
- total_findings: integer — number of findings across all checks
- vulnerability_rate: float — findings per scanned Python file
- findings: array — list of finding objects

3.4 Finding Object

Field	Type	Description
check_name	string	Name of the security check that fired
severity	string	HIGH, MEDIUM, or LOW
file_path	string	Relative path to the file containing the issue
line_number	integer	Line number where the issue was detected
detail	string	Plain English explanation of the finding